UQFFLearningAssessment_001.cpp

UQFF C++ Source Module

```
================================================================================
   ======== Header: UQFFLearningAssessment.h Description: C++ Module for UQFF Learning
Assessment — Research Student Edition (v2.0) This is the ninth module in a series of 500+
code files for the Universal Quantum Field Framework (UQFF) simulations. Covers ALL SEVEN
completed astrophysical systems in a unified research-student comparative framework: [0]
SGR 1745-2900 — Magnetar (compact, GC proximity, D(t) burst, 18 terms) [1] SGR 0501+4516
— Magnetar (compact, Perseus arm, D(t) burst, 16 terms)
```

Daniel T. Murphy • UQFF Research Consortium • 2026

Lines: 792 Size: 45,984 bytes

Module Description

```
================================================================================
================
```

Header: UQFFLearningAssessment.h

Description: C++ Module for UQFF Learning Assessment — Research Student Edition (v2.0)

This is the ninth module in a series of 500+ code files for the Universal Quantum

Field Framework (UQFF) simulations. Covers ALL SEVEN completed astrophysical systems

in a unified research-student comparative framework:

[0] SGR 1745-2900 — Magnetar (compact, GC proximity, D(t) burst, 18 terms)

[1] SGR 0501+4516 — Magnetar (compact, Perseus arm, D(t) burst, 16 terms)

[2] Sgr A* — SMBH (galactic center, M(t) accretion, QPO, 15 terms)

[3] Starbirth Tapestry — Young cluster (LMC NGC2014/2020, M(t) + wind, 12 terms)

[4] Westerlund 2 — Super cluster (Carina, M(t) + OB winds, 12 terms)

[5] Pillars of Creation— Star-forming pillars (Eagle Nebula M16, E(t) erosion, 12 terms)

[6] Rings of Relativity— Einstein ring (GAL-CLUS-022058s, H(z), L_t lensing, 12 terms)

Purpose: Research-student comparative framework for UQFF mastery assessment.

- Aggregates canonical parameters from all 7 completed C++ modules (Sessions 63-70)

- Computes g_base (Newtonian G*M/r^2), Ubi (buoyancy Tier-1), and advancement metrics

- Metrics: diversity (7/7 regimes), dynamic processes (7/10 unique), scale span

(~16.2 log-decades in r), coverage (7/7 planned modules complete)

- Advancement score: weighted mean of 4 metrics -> 0-100%

- Educational output: printStudentGuide() (unique physics per system, dynamic catalogue,

scale analysis), printComparativeAnalysis() (sorted g-table across all 7 systems)
- Full UQFF 2.0 Self-Expanding Framework (logging, dynamic_params, exportState,
cross_validate<>)
Integration: Designed for inclusion in base program 'ziqn233h.cpp' (not present here).
Instantiate class in main: UQFFLearningAssessment assess;
double adv = assess.compute_advancement();
assess.printStudentGuide();
assess.printComparativeAnalysis(1.0e6 * 3.156e7); // at t = 1 Myr
Key Features:
- Canonical parameters sourced directly from all 7 completed UQFF C++ modules
- 7 physical regimes: magnetar x2, SMBH, LMC cluster, MW super-cluster,
MW star-forming pillars, cosmological Einstein ring
- Scale range: r = 20 km (magnetar) to 10 kpc (Einstein ring) = 16.2 log-decades
- Mass range: M = 1.4 M_sun (magnetar) to 1e14 M_sun (galaxy cluster) = ~20 dex
- compute_advancement(): 4-metric weighted UQFF score (diversity, dynamics, scale, coverage)
- compute_g_base(int): Newtonian g = G*M/r^2 for system 0 through 6
- compute_Ubi(int): UQFF buoyancy Tier-1 = 0.5 * g_base
- compute_F_UBii(int, double): UQFF buoyancy Tier-2 at time t
- compute_Ub_i(int, double): UQFF buoyancy Tier-3 (external body) at time t
- printStudentGuide(): educational walkthrough + unique physics catalogue
- printComparativeAnalysis(double t): sorted g-table for all 7 systems
Author: Encoded by Grok (xAI), based on Daniel T. Murphy's UQFF manuscript.
Date: March 16, 2026 (Research Student Edition — replaced October 08, 2025 stub)
Copyright: Daniel T. Murphy, daniel.murphy00@gmail.com
==========
==========

Source Code

```
/**
 * ============================================================================================
 * Header: UQFFLearningAssessment.h
 *
 * Description: C++ Module for UQFF Learning Assessment — Research Student Edition (v2.0)
 *              This is the ninth module in a series of 500+ code files for the Universal Quantum
 *              Field Framework (UQFF) simulations. Covers ALL SEVEN completed astrophysical syste
 *              in a unified research-student comparative framework:
 *                [0] SGR 1745-2900      — Magnetar (compact, GC proximity, D(t) burst, 18 terms)
 *                [1] SGR 0501+4516      — Magnetar (compact, Perseus arm, D(t) burst, 16 terms)
```

```
 *               [2] Sgr A*            — SMBH (galactic center, M(t) accretion, QPO, 15 terms)
 *               [3] Starbirth Tapestry — Young cluster (LMC NGC2014/2020, M(t) + wind, 12 terms)
 *               [4] Westerlund 2       — Super cluster (Carina, M(t) + OB winds, 12 terms)
 *               [5] Pillars of Creation— Star-forming pillars (Eagle Nebula M16, E(t) erosion, 12
 *               [6] Rings of Relativity— Einstein ring (GAL-CLUS-022058s, H(z), L_t lensing, 12 t
 *
 * Purpose: Research-student comparative framework for UQFF mastery assessment.
 *          - Aggregates canonical parameters from all 7 completed C++ modules (Sessions 63-70)
 *          - Computes g_base (Newtonian G*M/r^2), Ubi (buoyancy Tier-1), and advancement metrics
 *          - Metrics: diversity (7/7 regimes), dynamic processes (7/10 unique), scale span
 *            (~16.2 log-decades in r), coverage (7/7 planned modules complete)
 *          - Advancement score: weighted mean of 4 metrics -&gt; 0-100%
 *          - Educational output: printStudentGuide() (unique physics per system, dynamic catalogue
 *            scale analysis), printComparativeAnalysis() (sorted g-table across all 7 systems)
 *          - Full UQFF 2.0 Self-Expanding Framework (logging, dynamic_params, exportState,
 *            cross_validate&lt;&gt;)
 *
 * Integration: Designed for inclusion in base program &#39;ziqn233h.cpp&#39; (not present here).
 *              Instantiate class in main: UQFFLearningAssessment assess;
 *              double adv = assess.compute_advancement();
 *              assess.printStudentGuide();
 *              assess.printComparativeAnalysis(1.0e6 * 3.156e7);  // at t = 1 Myr
 *
 * Key Features:
 *   - Canonical parameters sourced directly from all 7 completed UQFF C++ modules
 *   - 7 physical regimes: magnetar x2, SMBH, LMC cluster, MW super-cluster,
 *     MW star-forming pillars, cosmological Einstein ring
 *   - Scale range: r = 20 km (magnetar) to 10 kpc (Einstein ring) = 16.2 log-decades
 *   - Mass range: M = 1.4 M_sun (magnetar) to 1e14 M_sun (galaxy cluster) = ~20 dex
 *   - compute_advancement(): 4-metric weighted UQFF score (diversity, dynamics, scale, coverage)
 *   - compute_g_base(int): Newtonian g = G*M/r^2 for system 0 through 6
 *   - compute_Ubi(int): UQFF buoyancy Tier-1 = 0.5 * g_base
 *   - compute_F_UBii(int, double): UQFF buoyancy Tier-2 at time t
 *   - compute_Ub_i(int, double): UQFF buoyancy Tier-3 (external body) at time t
 *   - printStudentGuide(): educational walkthrough + unique physics catalogue
 *   - printComparativeAnalysis(double t): sorted g-table for all 7 systems
 *
 * Author: Encoded by Grok (xAI), based on Daniel T. Murphy&#39;s UQFF manuscript.
 * Date: March 16, 2026 (Research Student Edition — replaced October 08, 2025 stub)
 * Copyright: Daniel T. Murphy, daniel.murphy00@gmail.com
 * ================================================================================================
 */

#ifndef UQFF_LEARNING_ASSESSMENT_H
#define UQFF_LEARNING_ASSESSMENT_H

#include &lt;iostream&gt;
#include &lt;cmath&gt;
#include &lt;iomanip&gt;
#include &lt;string&gt;
#include &lt;limits&gt;
#include &lt;map&gt;
#include &lt;fstream&gt;
#include &lt;array&gt;
#include &lt;algorithm&gt;
```

```cpp

static constexpr double UQFF_LA_PI = 3.14159265358979323846;

// System index constants
static constexpr int SYS_SGR1745    = 0;
static constexpr int SYS_SGR0501    = 1;
static constexpr int SYS_SGR_A_STAR = 2;
static constexpr int SYS_TAPESTRY   = 3;
static constexpr int SYS_WESTERLUND = 4;
static constexpr int SYS_PILLARS    = 5;
static constexpr int SYS_RINGS      = 6;
static constexpr int NUM_SYSTEMS    = 7;

class UQFFLearningAssessment {
private:
    // ■■ Shared physical constants ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    double G;               // Gravitational constant (m^3 kg^-1 s^-2)
    double c_light;         // Speed of light (m/s)
    double hbar;            // Reduced Planck constant (J·s)
    double Lambda;          // Cosmological constant (m^-2)
    double M_sun;           // Solar mass (kg)
    double f_TRZ;           // UQFF time-reversal factor (shared, = 0.1)
    double rho_vac_UA;      // UA vacuum density (J/m^3)
    double rho_vac_SCm;     // SCm vacuum density (J/m^3)
    double t_Hubble;        // Hubble time (s)

    // ■■ UQFF buoyancy pipeline (CP3/PAPER_198 canonical) ■■■■■■■■■■■■■■■■■■■■■■■■■■■
    double beta_i;          // Buoyancy coupling constant (= 0.61)
    double omega_g;         // Galactic rotation rate (rad/s, = 7.3e-16)
    double U_UA;            // Unit charge aether parameter (C, = 1e-11)

    // ■■ Assessment metrics ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    double diversity_score;   // Fraction of distinct physical regimes covered (0-1)
    double dynamic_score;     // Fraction of distinct dynamic-physics processes covered (0-1)
    double scalability_score; // Scale-span coverage (0-1, based on log10 r range / 20)
    double coverage_score;    // Module completion fraction (completed / planned, 0-1)

    // ■■ System 0: SGR 1745-2900 (Magnetar — GC proximity, 0.92 pc from Sgr A*) ■■■■■■■■■■■■■
    double M_sgr1745;           // 1.4 M_sun (kg)
    double r_sgr1745;           // 20 km neutron-star radius (m)
    double B_sgr1745;           // Magnetic field ~1e14 T (near quantum-critical)
    double D0_sgr1745;          // D(t) burst amplitude (m/s^2)
    double omega_D_sgr1745;     // Burst angular freq: 2*pi/11 s (rad/s)
    double tau_D_sgr1745;       // Burst decay timescale: 3.5 yr (s)
    double M_ext_sgr1745;       // External body: Sgr A* ~4e6 M_sun (kg)
    double r_ext_sgr1745;       // Sgr A* distance: 0.92 pc = 2.83e16 m

    // ■■ System 1: SGR 0501+4516 (Magnetar — Perseus arm, 10 kpc from GC) ■■■■■■■■■■■■■■■■■■■■
    double M_sgr0501;           // 1.4 M_sun (kg)
    double r_sgr0501;           // 20 km neutron-star radius (m)
    double B_sgr0501;           // Magnetic field ~1.9e14 T (SGR0501 canonical)
    double D0_sgr0501;          // D(t) burst amplitude (m/s^2)
    double omega_D_sgr0501;     // Burst angular freq: 2*pi/15 s (2008 outburst cadence, rad/s)
    double tau_D_sgr0501;       // Burst decay timescale: 3.5 yr (s)
    double M_ext_sgr0501;       // External body: Sgr A* ~4e6 M_sun (kg)
```

```
121     double r_ext_sgr0501;       // Distance to GC: 10 kpc = 3.086e20 m
122
123     // ■■ System 2: Sgr A* (SMBH — galactic center) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
124     double M_smbh;              // Initial: 4e6 M_sun — M(t) grows by accretion (kg)
125     double r_smbh;              // Schwarzschild-region reference radius: 1.27e10 m
126     double M_dot_smbh;          // Accretion growth factor (dimensionless)
127     double tau_acc_smbh;        // Accretion timescale: 1 Myr (s)
128     double omega_D_smbh;        // QPO angular freq: 2*pi/1200 s (20-min Sgr A* QPO, rad/s)
129     double tau_D_smbh;          // QPO envelope decay: 3 yr (s)
130     double spin_smbh;           // Kerr spin factor (dimensionless, = 0.3)
131     double M_ext_smbh;          // External body: NSC ~2.5e7 M_sun (kg)
132     double r_ext_smbh;          // NSC distance: 3 pc = 9.258e16 m
133
134     // ■■ System 3: Starbirth Tapestry (LMC — NGC 2014/2020 young cluster) ■■■■■■■■■■■■■■■■■■■
135     double M_tapestry;          // Initial: 240 M_sun (kg)
136     double r_tapestry;          // 10 ly = 9.461e16 m
137     double M_dot_tapestry;      // SF mass-growth factor (~41.7, gas/stellar ratio)
138     double tau_SF_tapestry;     // Star-formation timescale: 5 Myr (s)
139     double rho_wind_tap;        // Wind density: 1e-21 kg/m^3
140     double v_wind_tap;          // Wind velocity: 2e6 m/s
141     double M_ext_tap;           // External body: R136 Tarantula core ~1e5 M_sun (kg)
142     double r_ext_tap;           // R136 distance: 650 ly = 6.15e18 m
143
144     // ■■ System 4: Westerlund 2 (super star cluster — Carina arm, ~4.16 kpc) ■■■■■■■■■■■■■■■■
145     double M_wd2;               // Initial: 30,000 M_sun (kg)
146     double r_wd2;               // 10 ly = 9.461e16 m
147     double M_dot_wd2;           // SF mass-growth factor (~3.33, gas/stellar ratio)
148     double tau_SF_wd2;          // Star-formation timescale: 2 Myr (s)
149     double rho_wind_wd2;        // Dense OB-star wind: 1e-20 kg/m^3
150     double v_wind_wd2;          // Wind velocity: 2e6 m/s
151     double M_ext_wd2;           // External body: Sgr A* ~4e6 M_sun (kg)
152     double r_ext_wd2;           // Wd2 to GC: 8 kpc = 2.47e20 m
153
154     // ■■ System 5: Pillars of Creation (Eagle Nebula M16 — Sagittarius arm) ■■■■■■■■■■■■■■■■■
155     double M_pillars;           // Initial: 10,100 M_sun (kg)
156     double r_pillars;           // 5 ly = 4.731e16 m
157     double M_dot_pillars;       // SF mass-growth factor (~0.99)
158     double tau_SF_pillars;      // Star-formation timescale: 1 Myr (s)
159     double E_0_pillars;         // Erosion amplitude: 0.1
160     double tau_E_pillars;       // Erosion timescale: 1 Myr (s)
161     double rho_wind_pil;        // Wind density: 1e-21 kg/m^3
162     double v_wind_pil;          // Wind velocity: 2e6 m/s
163     double M_ext_pil;           // External body: Sgr A* ~4e6 M_sun (kg)
164     double r_ext_pil;           // Pillars to GC: 6.85 kpc = 2.11e20 m
165
166     // ■■ System 6: Rings of Relativity (GAL-CLUS-022058s — z=0.5 Einstein ring) ■■■■■■■■■■■
167     double M_rings;             // Galaxy cluster: 1e14 M_sun; static (kg)
168     double r_rings;             // Einstein radius: 10 kpc = 3.086e20 m
169     double Hz_rings;            // H(z=0.5) = 2.42e-18 s^-1 (redshift-specific Hubble)
170     double L_factor_rings;      // D_LS/D_S = 0.67 (lensing geometry)
171     double z_rings;             // Lens redshift: 0.5
172     double rho_wind_rings;      // Intracluster wind density: 1e-21 kg/m^3
173     double v_wind_rings;        // ICM wind velocity: 2e6 m/s
174     double M_ext_rings;         // External body: Virgo Cluster ~1.2e15 M_sun (kg)
175     double r_ext_rings;         // Distance to Virgo: 16.5 Mpc = 5.09e23 m
```

```cpp

    // ■■ UQFF 2.0 Self-Expanding Framework ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    bool logging_enabled;
    std::map&lt;std::string, double&gt; dynamic_params;

    // ■■ Internal helpers (private) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    double g_base_for(int idx) const {
        switch (idx) {
            case SYS_SGR1745:    return (G * M_sgr1745)  / (r_sgr1745  * r_sgr1745);
            case SYS_SGR0501:    return (G * M_sgr0501)  / (r_sgr0501  * r_sgr0501);
            case SYS_SGR_A_STAR: return (G * M_smbh)     / (r_smbh     * r_smbh);
            case SYS_TAPESTRY:   return (G * M_tapestry) / (r_tapestry * r_tapestry);
            case SYS_WESTERLUND: return (G * M_wd2)      / (r_wd2      * r_wd2);
            case SYS_PILLARS:    return (G * M_pillars)  / (r_pillars  * r_pillars);
            case SYS_RINGS:      return (G * M_rings)    / (r_rings    * r_rings);
            default:             return 0.0;
        }
    }

    double get_M(int idx) const {
        switch (idx) {
            case SYS_SGR1745:    return M_sgr1745;
            case SYS_SGR0501:    return M_sgr0501;
            case SYS_SGR_A_STAR: return M_smbh;
            case SYS_TAPESTRY:   return M_tapestry;
            case SYS_WESTERLUND: return M_wd2;
            case SYS_PILLARS:    return M_pillars;
            case SYS_RINGS:      return M_rings;
            default:             return 0.0;
        }
    }

    double get_r(int idx) const {
        switch (idx) {
            case SYS_SGR1745:    return r_sgr1745;
            case SYS_SGR0501:    return r_sgr0501;
            case SYS_SGR_A_STAR: return r_smbh;
            case SYS_TAPESTRY:   return r_tapestry;
            case SYS_WESTERLUND: return r_wd2;
            case SYS_PILLARS:    return r_pillars;
            case SYS_RINGS:      return r_rings;
            default:             return 0.0;
        }
    }

    static const char* system_name(int idx) {
        static const char* names[NUM_SYSTEMS] = {
            &quot;SGR 1745-2900 (Magnetar)&quot;,
            &quot;SGR 0501+4516 (Magnetar)&quot;,
            &quot;Sgr A* (SMBH)&quot;,
            &quot;Starbirth Tapestry (LMC cluster)&quot;,
            &quot;Westerlund 2 (super cluster)&quot;,
            &quot;Pillars of Creation (Eagle Nebula M16)&quot;,
            &quot;Rings of Relativity (Einstein ring)&quot;
        };
```

```
            return (idx &gt;= 0 &amp;&amp; idx &lt; NUM_SYSTEMS) ? names[idx] : &quot;Unknown&quot;;
        }

        static const char* unique_physics_str(int idx) {
            static const char* phys[NUM_SYSTEMS] = {
                &quot;D(t) burst [omega_D=2pi/11s, tau_D=3.5yr]; BH tidal [2G*M_BH*r/r_BH^3]; 18 terms
                &quot;D(t) burst [omega_D=2pi/15s, 2008 cadence]; GW back-reaction; galactic tidal; 16
                &quot;M(t) accretion growth; QPO burst [omega_D=2pi/1200s, 20-min]; NSC tidal; magneti
                &quot;M(t) SF growth [tau=5Myr]; stellar wind [rho=1e-21]; R136 Tarantula ext 650ly; 1
                &quot;M(t) SF growth [tau=2Myr]; OB-star wind [rho=1e-20, richest]; Sgr A* ext 8 kpc; 
                &quot;M(t) SF growth + E(t) erosion [corr_E=1-E0*exp(-t/tau)]; 4 simultaneous co-actio
                &quot;Static M; H(z=0.5)=2.42e-18 s^-1 [H(z) — 1st cosmo module]; L_t=(GM/c^2*r)*Lfac
            };
            return (idx &gt;= 0 &amp;&amp; idx &lt; NUM_SYSTEMS) ? phys[idx] : &quot;Unknown&quot;;
        }

        static const char* regime_label(int idx) {
            static const char* labels[NUM_SYSTEMS] = {
                &quot;Compact / Nuclear-quantum&quot;,
                &quot;Compact / Nuclear-quantum&quot;,
                &quot;SMBH / Galactic center&quot;,
                &quot;Young stellar cluster / LMC&quot;,
                &quot;Super star cluster / MW Carina arm&quot;,
                &quot;Star-forming pillars / MW Sagittarius arm&quot;,
                &quot;Cosmological / Einstein ring&quot;
            };
            return (idx &gt;= 0 &amp;&amp; idx &lt; NUM_SYSTEMS) ? labels[idx] : &quot;Unknown&quot;;
        }

        // External body: M_ext and r_ext for each system
        void get_ext(int idx, double&amp; M_ext, double&amp; r_ext) const {
            switch (idx) {
                case SYS_SGR1745:    M_ext = M_ext_sgr1745; r_ext = r_ext_sgr1745; break;
                case SYS_SGR0501:    M_ext = M_ext_sgr0501; r_ext = r_ext_sgr0501; break;
                case SYS_SGR_A_STAR: M_ext = M_ext_smbh;    r_ext = r_ext_smbh;    break;
                case SYS_TAPESTRY:   M_ext = M_ext_tap;     r_ext = r_ext_tap;     break;
                case SYS_WESTERLUND: M_ext = M_ext_wd2;     r_ext = r_ext_wd2;     break;
                case SYS_PILLARS:    M_ext = M_ext_pil;     r_ext = r_ext_pil;     break;
                case SYS_RINGS:      M_ext = M_ext_rings;   r_ext = r_ext_rings;   break;
                default:             M_ext = 0.0;           r_ext = 1.0;           break;
            }
        }

    public:
        // ■■ Constructor ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

        UQFFLearningAssessment() { initializeDefaults(); }
        ~UQFFLearningAssessment() {}

        // ■■ Initialization ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        void initializeDefaults() {
            // Shared constants
            G           = 6.6743e-11;
            c_light     = 3.0e8;
            hbar        = 1.0546e-34;
            Lambda      = 1.1e-52;
```

```
286         M_sun       = 1.989e30;
287         f_TRZ       = 0.1;
288         rho_vac_UA  = 7.09e-36;
289         rho_vac_SCm = 7.09e-37;
290         t_Hubble    = 13.8e9 * 3.15576e7;
291
292         // UQFF buoyancy pipeline (CP3/PAPER_198 canonical)
293         beta_i  = 0.61;
294         omega_g = 7.3e-16;
295         U_UA    = 1e-11;
296
297         // Assessment metrics
298         // diversity:   7 distinct physical regimes covered out of 7
299         // dynamic:     7 unique time-dependent physics processes introduced
300         // scalability: log10(r_max/r_min) = log10(3.086e20 / 2e4) ~= 16.2 / 20 = 0.81
301         // coverage:    7/7 planned C++ modules complete
302         diversity_score   = 7.0 / 7.0;
303         dynamic_score     = 7.0 / 10.0;
304         scalability_score = 16.2 / 20.0;
305         coverage_score    = 7.0 / 7.0;
306
307         // ■■ System 0: SGR 1745-2900 (Magnetar — 0.92 pc from Sgr A*) ■■■■■■■■■■■■■■■■■■■■
308         M_sgr1745       = 1.4 * M_sun;                    // 1.4 M_sun
309         r_sgr1745       = 20.0e3;                          // 20 km
310         B_sgr1745       = 1.0e14;                          // Near quantum-critical field (T)
311         D0_sgr1745      = 1.0e10;                          // Burst amplitude (m/s^2)
312         omega_D_sgr1745 = 2.0 * UQFF_LA_PI / 11.0;        // 11 s burst period (rad/s)
313         tau_D_sgr1745   = 3.5 * 3.15576e7;                // 3.5 yr burst decay (s)
314         M_ext_sgr1745   = 4.0e6 * M_sun;                  // Sgr A* (kg)
315         r_ext_sgr1745   = 2.83e16;                         // 0.92 pc (m)
316
317         // ■■ System 1: SGR 0501+4516 (Magnetar — Perseus arm) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
318         M_sgr0501       = 1.4 * M_sun;                    // 1.4 M_sun
319         r_sgr0501       = 20.0e3;                          // 20 km
320         B_sgr0501       = 1.9e14;                          // SGR0501 canonical field (T)
321         D0_sgr0501      = 1.0e10;                          // Burst amplitude (m/s^2)
322         omega_D_sgr0501 = 2.0 * UQFF_LA_PI / 15.0;        // 15 s burst cadence (2008 outburst, ra
323         tau_D_sgr0501   = 3.5 * 3.15576e7;                // 3.5 yr burst decay (s)
324         M_ext_sgr0501   = 4.0e6 * M_sun;                  // Sgr A* (kg)
325         r_ext_sgr0501   = 3.086e20;                        // 10 kpc (m)
326
327         // ■■ System 2: Sgr A* (SMBH — galactic center) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
328         M_smbh          = 4.0e6 * M_sun;                  // 4e6 M_sun initial
329         r_smbh          = 1.27e10;                         // Schwarzschild-region radius (m)
330         M_dot_smbh      = 1.0e-4;                          // Accretion growth factor
331         tau_acc_smbh    = 1.0e6 * 3.15576e7;              // 1 Myr accretion timescale (s)
332         omega_D_smbh    = 2.0 * UQFF_LA_PI / 1200.0;      // 20-min QPO (rad/s)
333         tau_D_smbh      = 3.0 * 3.15576e7;                // 3-yr QPO envelope decay (s)
334         spin_smbh       = 0.3;                             // Kerr spin parameter
335         M_ext_smbh      = 2.5e7 * M_sun;                  // NSC ~2.5e7 M_sun (kg)
336         r_ext_smbh      = 9.258e16;                        // NSC: 3 pc (m)
337
338         // ■■ System 3: Starbirth Tapestry (LMC — NGC 2014/2020) ■■■■■■■■■■■■■■■■■■■■■■■■
339         M_tapestry      = 240.0 * M_sun;                  // 240 M_sun initial
340         r_tapestry      = 9.461e16;                        // 10 ly (m)
```

```
341         M_dot_tapestry  = 1.0e4 / 240.0;                  // SF growth factor ~41.7
342         tau_SF_tapestry = 5.0e6 * 3.156e7;                // 5 Myr (s)
343         rho_wind_tap    = 1.0e-21;                         // Wind density (kg/m^3)
344         v_wind_tap      = 2.0e6;                           // Wind velocity (m/s)
345         M_ext_tap       = 1.0e5 * M_sun;                  // R136 Tarantula core (kg)
346         r_ext_tap       = 6.15e18;                         // 650 ly (m)
347
348         // ■■ System 4: Westerlund 2 (super star cluster — Carina arm) ■■■■■■■■■■■■■■■■■■■■■
349         M_wd2           = 30000.0 * M_sun;                // 30,000 M_sun initial
350         r_wd2           = 9.461e16;                        // 10 ly (m)
351         M_dot_wd2       = 1.0e5 / 30000.0;                // SF growth factor ~3.33
352         tau_SF_wd2      = 2.0e6 * 3.156e7;                // 2 Myr (s)
353         rho_wind_wd2    = 1.0e-20;                         // Dense OB-star wind (kg/m^3)
354         v_wind_wd2      = 2.0e6;                           // Wind velocity (m/s)
355         M_ext_wd2       = 4.0e6 * M_sun;                  // Sgr A* (kg)
356         r_ext_wd2       = 2.47e20;                         // 8 kpc (m)
357
358         // ■■ System 5: Pillars of Creation (Eagle Nebula M16 — Sagittarius arm) ■■■■■■■■■■
359         M_pillars       = 10100.0 * M_sun;                // 10,100 M_sun initial
360         r_pillars       = 4.731e16;                        // 5 ly (m)
361         M_dot_pillars   = 1.0e4 / 10100.0;                // SF growth factor ~0.99
362         tau_SF_pillars  = 1.0e6 * 3.156e7;                // 1 Myr (s)
363         E_0_pillars     = 0.1;                             // Erosion amplitude
364         tau_E_pillars   = 1.0e6 * 3.156e7;                // 1 Myr erosion timescale (s)
365         rho_wind_pil    = 1.0e-21;                         // Wind density (kg/m^3)
366         v_wind_pil      = 2.0e6;                           // Wind velocity (m/s)
367         M_ext_pil       = 4.0e6 * M_sun;                  // Sgr A* (kg)
368         r_ext_pil       = 2.11e20;                         // 6.85 kpc (m)
369
370         // ■■ System 6: Rings of Relativity (GAL-CLUS-022058s — z=0.5 Einstein ring) ■■■■■■
371         M_rings         = 1.0e14 * M_sun;                 // Galaxy cluster: static (kg)
372         r_rings         = 3.086e20;                        // Einstein radius: 10 kpc (m)
373         Hz_rings        = 2.42e-18;                        // H(z=0.5) (s^-1)
374         L_factor_rings  = 0.67;                            // D_LS/D_S lensing geometry
375         z_rings         = 0.5;                             // Lens redshift
376         rho_wind_rings  = 1.0e-21;                         // Intracluster wind (kg/m^3)
377         v_wind_rings    = 2.0e6;                           // ICM wind velocity (m/s)
378         M_ext_rings     = 2.387e45;                        // Virgo Cluster ~1.2e15 M_sun (kg)
379         r_ext_rings     = 5.09e23;                         // 16.5 Mpc (m)
380
381         logging_enabled = false;
382     }
383
384     // ■■ Core assessment computations ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
385
386     // Advancement score (0-100%): weighted mean of 4 metrics
387     double compute_advancement() const {
388         double score = (diversity_score + dynamic_score + scalability_score + coverage_score)
389                        / 4.0 * 100.0;
390         if (logging_enabled)
391             std::cout &lt;&lt; &quot;[LOG] compute_advancement()=&quot; &lt;&lt; score
392                       &lt;&lt; &quot;  div=&quot; &lt;&lt; diversity_score &lt;&lt; &quot; dyn=&quo
393                       &lt;&lt; &quot; scale=&quot; &lt;&lt; scalability_score &lt;&lt; &quot; cov=&
394         return score;
395     }
```

```
396
397     // Newtonian g = G*M/r^2 for system idx (0 = SGR1745 ... 6 = RINGS)
398     double compute_g_base(int idx) const {
399         if (idx &lt; 0 || idx &gt;= NUM_SYSTEMS) {
400             std::cerr &lt;&lt; &quot;Error: system index &quot; &lt;&lt; idx
401                       &lt;&lt; &quot; out of range [0,&quot; &lt;&lt; (NUM_SYSTEMS - 1) &lt;&lt; &q
402             return 0.0;
403         }
404         double g = g_base_for(idx);
405         if (logging_enabled)
406             std::cout &lt;&lt; &quot;[LOG] g_base[&quot; &lt;&lt; idx &lt;&lt; &quot; &quot; &lt;&
407                       &lt;&lt; &quot;]=&quot; &lt;&lt; std::scientific &lt;&lt; g &lt;&lt; &quot; m
408         return g;
409     }
410
411     // log10(r_max / r_min) across all 7 systems
412     double compute_scale_range() const {
413         double r_min = r_sgr1745;   // 20 km — magnetar surface
414         double r_max = r_rings;     // 10 kpc — Einstein radius
415         double span  = std::log10(r_max / r_min);
416         if (logging_enabled)
417             std::cout &lt;&lt; &quot;[LOG] scale_range: r_min=&quot; &lt;&lt; r_min
418                       &lt;&lt; &quot; r_max=&quot; &lt;&lt; r_max &lt;&lt; &quot; log10=&quot; &lt
419         return span;
420     }
421
422     // UQFF buoyancy Tier-1: term_Ubi = 0.5 * g_base (CP3/PAPER_198 static half-gravity)
423     double compute_Ubi(int idx) const {
424         return 0.5 * g_base_for(idx);
425     }
426
427     // UQFF buoyancy Tier-2: term_F_UBii = -beta_i * g * omega_g * (M/r) * U_UA * cos(pi*t)
428     double compute_F_UBii(int idx, double t) const {
429         double g  = g_base_for(idx);
430         double Mt = get_M(idx);
431         double ri = get_r(idx);
432         return -beta_i * g * omega_g * (Mt / ri) * U_UA * std::cos(UQFF_LA_PI * t);
433     }
434
435     // UQFF buoyancy Tier-3: term_Ub_i = -beta_i * g * omega_g * (M_ext/r_ext) * U_UA * cos(pi*t)
436     double compute_Ub_i(int idx, double t) const {
437         double g = g_base_for(idx);
438         double M_ext = 0.0, r_ext = 1.0;
439         get_ext(idx, M_ext, r_ext);
440         return -beta_i * g * omega_g * (M_ext / r_ext) * U_UA * std::cos(UQFF_LA_PI * t);
441     }
442
443     // ■■ Comparative analysis output ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
444
445     // Print sorted g-value table for all 7 systems at time t (seconds)
446     void printComparativeAnalysis(double t = 0.0, std::ostream&amp; os = std::cout) const {
447         os &lt;&lt; &quot;\n============================================================\n
448         os &lt;&lt; &quot;  UQFF COMPARATIVE ANALYSIS — Research Student Module\n&quot;;
449         os &lt;&lt; &quot;  t = &quot; &lt;&lt; std::scientific &lt;&lt; std::setprecision(4) &lt;&
450         os &lt;&lt; &quot;============================================================\n&qu
```

```
451
452         // Header row
453         os &lt;&lt; std::left
454            &lt;&lt; std::setw(4)  &lt;&lt; &quot;Idx&quot;
455            &lt;&lt; std::setw(44) &lt;&lt; &quot;System&quot;
456            &lt;&lt; std::setw(15) &lt;&lt; &quot;g_base(m/s^2)&quot;
457            &lt;&lt; std::setw(15) &lt;&lt; &quot;Ubi(m/s^2)&quot;
458            &lt;&lt; std::setw(15) &lt;&lt; &quot;r(m)&quot;
459            &lt;&lt; &quot;M(kg)&quot; &lt;&lt; &quot;\n&quot;;
460         os &lt;&lt; std::string(110, &#39;-&#39;) &lt;&lt; &quot;\n&quot;;
461
462         // Sort by g_base descending
463         std::array&lt;int, NUM_SYSTEMS&gt; order;
464         for (int i = 0; i &lt; NUM_SYSTEMS; ++i) order[i] = i;
465         std::sort(order.begin(), order.end(),
466                   [&amp;](int a, int b){ return g_base_for(a) &gt; g_base_for(b); });
467
468         for (int k = 0; k &lt; NUM_SYSTEMS; ++k) {
469             int i   = order[k];
470             double gb  = g_base_for(i);
471             double ubi = compute_Ubi(i);
472             os &lt;&lt; std::left
473                &lt;&lt; std::setw(4)  &lt;&lt; i
474                &lt;&lt; std::setw(44) &lt;&lt; system_name(i)
475                &lt;&lt; std::scientific &lt;&lt; std::setprecision(4)
476                &lt;&lt; std::setw(15) &lt;&lt; gb
477                &lt;&lt; std::setw(15) &lt;&lt; ubi
478                &lt;&lt; std::setw(15) &lt;&lt; get_r(i)
479                &lt;&lt; get_M(i) &lt;&lt; &quot;\n&quot;;
480         }
481         os &lt;&lt; std::string(110, &#39;-&#39;) &lt;&lt; &quot;\n&quot;;
482         os &lt;&lt; &quot;  Scale range (log10 r_max/r_min): &quot; &lt;&lt; std::fixed &lt;&lt; s
483            &lt;&lt; compute_scale_range() &lt;&lt; &quot; decades\n&quot;;
484         os &lt;&lt; &quot;  Advancement score: &quot; &lt;&lt; std::fixed &lt;&lt; std::setprecisi
485            &lt;&lt; compute_advancement() &lt;&lt; &quot;%\n&quot;;
486         os &lt;&lt; &quot;==========================================================================\n\n
487     }
488
489     // Educational walkthrough: unique physics per system, dynamic catalogue, scale analysis
490     void printStudentGuide(std::ostream&amp; os = std::cout) const {
491         os &lt;&lt; &quot;\n============================================================\n&quot
492         os &lt;&lt; &quot;  UQFF Research Student Guide — 7-System MUGE Series\n&quot;;
493         os &lt;&lt; &quot;  Universal Quantum Field Framework | CP3/PAPER_198 Buoyancy\n&quot;;
494         os &lt;&lt; &quot;============================================================\n\n&quot
495
496         os &lt;&lt; &quot;CORE ARCHITECTURE (applies to ALL systems)\n&quot;;
497         os &lt;&lt; &quot;--------------------------------------\n&quot;;
498         os &lt;&lt; &quot;  MUGE g(r,t) = sum of terms:\n&quot;;
499         os &lt;&lt; &quot;    term1 = g_base * (1+Hz*t) * (1-B/B_crit) * [unique corr per system]\
500         os &lt;&lt; &quot;    term2 = compute_Ug (Ug1+Ug2+Ug3+Ug4) * (1+f_TRZ)\n&quot;;
501         os &lt;&lt; &quot;    term3 = Lambda*c^2/3  (cosmological constant)\n&quot;;
502         os &lt;&lt; &quot;    term4 = EM correction with [UA] vacuum\n&quot;;
503         os &lt;&lt; &quot;    term_q, term_fluid, term_osc, term_DM, term_wind (shared)\n&quot;;
504         os &lt;&lt; &quot;  Buoyancy Pipeline (3 tiers, CP3/PAPER_198):\n&quot;;
505         os &lt;&lt; &quot;    Tier-1  term_Ubi    = 0.5 * g_base          [static half-gravity]\n
```

```
506         os &lt;&lt; &quot;    Tier-2  term_F_UBii = -b_i * g * w_g * (M/r) * [UA] * cos(pi*t)\n&quo
507         os &lt;&lt; &quot;    Tier-3  term_Ub_i   = -b_i * g * w_g * (M_ext/r_ext) * [UA] * cos(pi
508         os &lt;&lt; std::scientific &lt;&lt; std::setprecision(3);
509         os &lt;&lt; &quot;    b_i=&quot; &lt;&lt; beta_i &lt;&lt; &quot;  w_g=&quot; &lt;&lt; omega
510
511         for (int i = 0; i &lt; NUM_SYSTEMS; ++i) {
512             os &lt;&lt; &quot;System [&quot; &lt;&lt; i &lt;&lt; &quot;] &quot; &lt;&lt; system_nam
513             os &lt;&lt; &quot;  Regime   : &quot; &lt;&lt; regime_label(i) &lt;&lt; &quot;\n&quot;;
514             os &lt;&lt; std::scientific &lt;&lt; std::setprecision(4);
515             os &lt;&lt; &quot;  M        : &quot; &lt;&lt; get_M(i) &lt;&lt; &quot; kg  (&quot;;
516                &lt;&lt; std::fixed &lt;&lt; std::setprecision(0) &lt;&lt; (get_M(i) / M_sun) &lt;&
517             os &lt;&lt; std::scientific &lt;&lt; std::setprecision(4);
518             os &lt;&lt; &quot;  r        : &quot; &lt;&lt; get_r(i) &lt;&lt; &quot; m\n&quot;;
519             os &lt;&lt; &quot;  g_base   : &quot; &lt;&lt; g_base_for(i) &lt;&lt; &quot; m/s^2\n&qu
520             os &lt;&lt; &quot;  Ubi      : &quot; &lt;&lt; compute_Ubi(i) &lt;&lt; &quot; m/s^2\n&
521             os &lt;&lt; &quot;  Unique   : &quot; &lt;&lt; unique_physics_str(i) &lt;&lt; &quot;\n
522         }
523
524         os &lt;&lt; &quot;DYNAMIC PROCESSES CATALOGUE (7 unique time-dependent physics introduced)
525         os &lt;&lt; &quot;-------------------------------------------------------------------------
526         os &lt;&lt; &quot;  1. D(t) burst   = D0*cos(omega_D*t)*exp(-t/tau_D)  [Systems 0,1 — magne
527         os &lt;&lt; &quot;  2. QPO burst    = D0*cos(omega_D*t)*exp(-t/tau_D), omega_D=2pi/1200s [S
528         os &lt;&lt; &quot;  3. M(t) growth  = M0*(1+Mdot*exp(-t/tau_SF))       [Systems 2,3,4,5 — a
529         os &lt;&lt; &quot;  4. E(t) erosion = E0*exp(-t/tau_E) -&gt; corr_E=1-E(t)[System 5 — Pilla
530         os &lt;&lt; &quot;  5. H(z) Hubble  = H0*sqrt(Om*(1+z)^3+OL)           [System 6 — Rings, 2
531         os &lt;&lt; &quot;  6. L_t lensing  = (G*M/c^2*r)*L_fac -&gt; corr_L=1+L_t [System 6 — Ring
532         os &lt;&lt; &quot;  7. Tidal force  = 2*G*M_ext*r/r_ext^3              [Systems 0,1,2 — co
533
534         os &lt;&lt; &quot;SCALE ANALYSIS\n&quot;;
535         os &lt;&lt; &quot;--------------\n&quot;;
536         os &lt;&lt; std::scientific &lt;&lt; std::setprecision(3);
537         os &lt;&lt; &quot;  r_min = &quot; &lt;&lt; r_sgr1745 &lt;&lt; &quot; m  (magnetar, 20 km -
538         os &lt;&lt; &quot;  r_max = &quot; &lt;&lt; r_rings   &lt;&lt; &quot; m  (Einstein ring, 10
539         os &lt;&lt; std::fixed &lt;&lt; std::setprecision(2);
540         os &lt;&lt; &quot;  log10(r_max/r_min) = &quot; &lt;&lt; compute_scale_range() &lt;&lt; &qu
541
542         os &lt;&lt; &quot;ADVANCEMENT METRICS\n&quot;;
543         os &lt;&lt; &quot;-------------------\n&quot;;
544         os &lt;&lt; std::fixed &lt;&lt; std::setprecision(4);
545         os &lt;&lt; &quot;  diversity_score   = &quot; &lt;&lt; diversity_score
546            &lt;&lt; &quot;  (7/7 distinct physical regimes)\n&quot;;
547         os &lt;&lt; &quot;  dynamic_score     = &quot; &lt;&lt; dynamic_score
548            &lt;&lt; &quot;  (7/10 unique dynamic-physics processes)\n&quot;;
549         os &lt;&lt; &quot;  scalability_score = &quot; &lt;&lt; scalability_score
550            &lt;&lt; &quot;  (16.2/20 log-decades of r coverage)\n&quot;;
551         os &lt;&lt; &quot;  coverage_score    = &quot; &lt;&lt; coverage_score
552            &lt;&lt; &quot;  (7/7 planned C++ modules complete)\n&quot;;
553         os &lt;&lt; std::fixed &lt;&lt; std::setprecision(1);
554         os &lt;&lt; &quot;  OVERALL ADVANCEMENT : &quot; &lt;&lt; compute_advancement() &lt;&lt; &
555         os &lt;&lt; &quot;================================================================\n\n&quot
556     }
557
558     // Compact parameter dump for all 7 systems
559     void printParameters(std::ostream&amp; os = std::cout) const {
560         os &lt;&lt; std::fixed &lt;&lt; std::setprecision(3);
```

```
        os &lt;&lt; &quot;UQFFLearningAssessment (Research Student Edition)\n&quot;;
        os &lt;&lt; &quot;  G=&quot; &lt;&lt; G &lt;&lt; &quot;  c_light=&quot; &lt;&lt; c_light &
        os &lt;&lt; &quot;  beta_i=&quot; &lt;&lt; beta_i &lt;&lt; &quot;  omega_g=&quot; &lt;&lt;
        for (int i = 0; i &lt; NUM_SYSTEMS; ++i) {
            os &lt;&lt; &quot;  [&quot; &lt;&lt; i &lt;&lt; &quot;] &quot; &lt;&lt; system_name(i)
               &lt;&lt; &quot;  M=&quot; &lt;&lt; std::scientific &lt;&lt; get_M(i)
               &lt;&lt; &quot;  r=&quot; &lt;&lt; get_r(i)
               &lt;&lt; &quot;  g_base=&quot; &lt;&lt; g_base_for(i) &lt;&lt; &quot;\n&quot;;
        }
        os &lt;&lt; &quot;  advancement=&quot; &lt;&lt; std::fixed &lt;&lt; std::setprecision(1) &
    }

    // Example: print guide + comparative analysis at t = 1 Myr
    double exampleAt1Myr() const {
        const double t = 1.0e6 * 3.156e7;
        printStudentGuide();
        printComparativeAnalysis(t);
        return compute_advancement();
    }

    // ■■ setVariable / getVariable / addToVariable / subtractFromVariable ■■■■■■
    bool setVariable(const std::string&amp; varName, double newValue) {
        if      (varName == &quot;G&quot;)              G              = newValue;
        else if (varName == &quot;c_light&quot;)        c_light        = newValue;
        else if (varName == &quot;hbar&quot;)           hbar           = newValue;
        else if (varName == &quot;Lambda&quot;)         Lambda         = newValue;
        else if (varName == &quot;f_TRZ&quot;)          f_TRZ          = newValue;
        else if (varName == &quot;beta_i&quot;)         beta_i         = newValue;
        else if (varName == &quot;omega_g&quot;)        omega_g        = newValue;
        else if (varName == &quot;U_UA&quot;)           U_UA           = newValue;
        else if (varName == &quot;diversity_score&quot;)   diversity_score   = newValue;
        else if (varName == &quot;dynamic_score&quot;)     dynamic_score     = newValue;
        else if (varName == &quot;scalability_score&quot;) scalability_score = newValue;
        else if (varName == &quot;coverage_score&quot;)    coverage_score    = newValue;
        // SGR1745
        else if (varName == &quot;M_sgr1745&quot;)      M_sgr1745      = newValue;
        else if (varName == &quot;r_sgr1745&quot;)      r_sgr1745      = newValue;
        else if (varName == &quot;B_sgr1745&quot;)      B_sgr1745      = newValue;
        else if (varName == &quot;D0_sgr1745&quot;)     D0_sgr1745     = newValue;
        else if (varName == &quot;M_ext_sgr1745&quot;)  M_ext_sgr1745  = newValue;
        else if (varName == &quot;r_ext_sgr1745&quot;)  r_ext_sgr1745  = newValue;
        // SGR0501
        else if (varName == &quot;M_sgr0501&quot;)      M_sgr0501      = newValue;
        else if (varName == &quot;r_sgr0501&quot;)      r_sgr0501      = newValue;
        else if (varName == &quot;B_sgr0501&quot;)      B_sgr0501      = newValue;
        else if (varName == &quot;D0_sgr0501&quot;)     D0_sgr0501     = newValue;
        else if (varName == &quot;M_ext_sgr0501&quot;)  M_ext_sgr0501  = newValue;
        else if (varName == &quot;r_ext_sgr0501&quot;)  r_ext_sgr0501  = newValue;
        // SMBH Sgr A*
        else if (varName == &quot;M_smbh&quot;)         M_smbh         = newValue;
        else if (varName == &quot;r_smbh&quot;)         r_smbh         = newValue;
        else if (varName == &quot;M_dot_smbh&quot;)     M_dot_smbh     = newValue;
        else if (varName == &quot;spin_smbh&quot;)      spin_smbh      = newValue;
        else if (varName == &quot;M_ext_smbh&quot;)     M_ext_smbh     = newValue;
        else if (varName == &quot;r_ext_smbh&quot;)     r_ext_smbh     = newValue;
```

```
        // Tapestry
        else if (varName == &quot;M_tapestry&quot;)      M_tapestry      = newValue;
        else if (varName == &quot;r_tapestry&quot;)      r_tapestry      = newValue;
        else if (varName == &quot;rho_wind_tap&quot;)    rho_wind_tap    = newValue;
        else if (varName == &quot;v_wind_tap&quot;)      v_wind_tap      = newValue;
        else if (varName == &quot;M_ext_tap&quot;)       M_ext_tap       = newValue;
        else if (varName == &quot;r_ext_tap&quot;)       r_ext_tap       = newValue;
        // Westerlund 2
        else if (varName == &quot;M_wd2&quot;)          M_wd2           = newValue;
        else if (varName == &quot;r_wd2&quot;)          r_wd2           = newValue;
        else if (varName == &quot;rho_wind_wd2&quot;)    rho_wind_wd2    = newValue;
        else if (varName == &quot;v_wind_wd2&quot;)      v_wind_wd2      = newValue;
        else if (varName == &quot;M_ext_wd2&quot;)       M_ext_wd2       = newValue;
        else if (varName == &quot;r_ext_wd2&quot;)       r_ext_wd2       = newValue;
        // Pillars
        else if (varName == &quot;M_pillars&quot;)       M_pillars       = newValue;
        else if (varName == &quot;r_pillars&quot;)       r_pillars       = newValue;
        else if (varName == &quot;E_0_pillars&quot;)     E_0_pillars     = newValue;
        else if (varName == &quot;rho_wind_pil&quot;)    rho_wind_pil    = newValue;
        else if (varName == &quot;v_wind_pil&quot;)      v_wind_pil      = newValue;
        else if (varName == &quot;M_ext_pil&quot;)       M_ext_pil       = newValue;
        else if (varName == &quot;r_ext_pil&quot;)       r_ext_pil       = newValue;
        // Rings
        else if (varName == &quot;M_rings&quot;)        M_rings         = newValue;
        else if (varName == &quot;r_rings&quot;)        r_rings         = newValue;
        else if (varName == &quot;Hz_rings&quot;)       Hz_rings        = newValue;
        else if (varName == &quot;L_factor_rings&quot;) L_factor_rings  = newValue;
        else if (varName == &quot;z_rings&quot;)        z_rings         = newValue;
        else if (varName == &quot;M_ext_rings&quot;)    M_ext_rings     = newValue;
        else if (varName == &quot;r_ext_rings&quot;)    r_ext_rings     = newValue;
        else {
            std::cerr &lt;&lt; &quot;Error: Unknown variable &#39;&quot; &lt;&lt; varName &lt;&lt;
            return false;
        }
        if (logging_enabled)
            std::cout &lt;&lt; &quot;[LOG] setVariable: &quot; &lt;&lt; varName &lt;&lt; &quot; = &
        return true;
    }

    bool addToVariable(const std::string&amp; varName, double delta) {
        return setVariable(varName, getVariable(varName) + delta);
    }

    bool subtractFromVariable(const std::string&amp; varName, double delta) {
        return addToVariable(varName, -delta);
    }

    double getVariable(const std::string&amp; varName) const {
        if      (varName == &quot;G&quot;)              return G;
        else if (varName == &quot;c_light&quot;)        return c_light;
        else if (varName == &quot;hbar&quot;)           return hbar;
        else if (varName == &quot;Lambda&quot;)         return Lambda;
        else if (varName == &quot;f_TRZ&quot;)          return f_TRZ;
        else if (varName == &quot;beta_i&quot;)         return beta_i;
        else if (varName == &quot;omega_g&quot;)        return omega_g;
```

```
671         else if (varName == &quot;U_UA&quot;)           return U_UA;
672         else if (varName == &quot;diversity_score&quot;)   return diversity_score;
673         else if (varName == &quot;dynamic_score&quot;)     return dynamic_score;
674         else if (varName == &quot;scalability_score&quot;) return scalability_score;
675         else if (varName == &quot;coverage_score&quot;)    return coverage_score;
676         else if (varName == &quot;M_sgr1745&quot;)       return M_sgr1745;
677         else if (varName == &quot;r_sgr1745&quot;)       return r_sgr1745;
678         else if (varName == &quot;B_sgr1745&quot;)       return B_sgr1745;
679         else if (varName == &quot;D0_sgr1745&quot;)      return D0_sgr1745;
680         else if (varName == &quot;M_ext_sgr1745&quot;)   return M_ext_sgr1745;
681         else if (varName == &quot;r_ext_sgr1745&quot;)   return r_ext_sgr1745;
682         else if (varName == &quot;M_sgr0501&quot;)       return M_sgr0501;
683         else if (varName == &quot;r_sgr0501&quot;)       return r_sgr0501;
684         else if (varName == &quot;B_sgr0501&quot;)       return B_sgr0501;
685         else if (varName == &quot;D0_sgr0501&quot;)      return D0_sgr0501;
686         else if (varName == &quot;M_ext_sgr0501&quot;)   return M_ext_sgr0501;
687         else if (varName == &quot;r_ext_sgr0501&quot;)   return r_ext_sgr0501;
688         else if (varName == &quot;M_smbh&quot;)          return M_smbh;
689         else if (varName == &quot;r_smbh&quot;)          return r_smbh;
690         else if (varName == &quot;M_dot_smbh&quot;)      return M_dot_smbh;
691         else if (varName == &quot;spin_smbh&quot;)       return spin_smbh;
692         else if (varName == &quot;M_ext_smbh&quot;)      return M_ext_smbh;
693         else if (varName == &quot;r_ext_smbh&quot;)      return r_ext_smbh;
694         else if (varName == &quot;M_tapestry&quot;)      return M_tapestry;
695         else if (varName == &quot;r_tapestry&quot;)      return r_tapestry;
696         else if (varName == &quot;rho_wind_tap&quot;)    return rho_wind_tap;
697         else if (varName == &quot;v_wind_tap&quot;)      return v_wind_tap;
698         else if (varName == &quot;M_ext_tap&quot;)       return M_ext_tap;
699         else if (varName == &quot;r_ext_tap&quot;)       return r_ext_tap;
700         else if (varName == &quot;M_wd2&quot;)           return M_wd2;
701         else if (varName == &quot;r_wd2&quot;)           return r_wd2;
702         else if (varName == &quot;rho_wind_wd2&quot;)    return rho_wind_wd2;
703         else if (varName == &quot;v_wind_wd2&quot;)      return v_wind_wd2;
704         else if (varName == &quot;M_ext_wd2&quot;)       return M_ext_wd2;
705         else if (varName == &quot;r_ext_wd2&quot;)       return r_ext_wd2;
706         else if (varName == &quot;M_pillars&quot;)       return M_pillars;
707         else if (varName == &quot;r_pillars&quot;)       return r_pillars;
708         else if (varName == &quot;E_0_pillars&quot;)     return E_0_pillars;
709         else if (varName == &quot;rho_wind_pil&quot;)    return rho_wind_pil;
710         else if (varName == &quot;v_wind_pil&quot;)      return v_wind_pil;
711         else if (varName == &quot;M_ext_pil&quot;)       return M_ext_pil;
712         else if (varName == &quot;r_ext_pil&quot;)       return r_ext_pil;
713         else if (varName == &quot;M_rings&quot;)         return M_rings;
714         else if (varName == &quot;r_rings&quot;)         return r_rings;
715         else if (varName == &quot;Hz_rings&quot;)        return Hz_rings;
716         else if (varName == &quot;L_factor_rings&quot;)  return L_factor_rings;
717         else if (varName == &quot;z_rings&quot;)         return z_rings;
718         else if (varName == &quot;M_ext_rings&quot;)     return M_ext_rings;
719         else if (varName == &quot;r_ext_rings&quot;)     return r_ext_rings;
720         else {
721             std::cerr &lt;&lt; &quot;Error: Unknown variable &#39;&quot; &lt;&lt; varName &lt;&lt;
722             return 0.0;
723         }
724     }
725
```

```
// ■■ UQFF 2.0 Self-Expanding Framework ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

void setEnableLogging(bool enabled) { logging_enabled = enabled; }
bool getLoggingEnabled() const      { return logging_enabled; }

void setDynamicParameter(const std::string&amp; key, double value) {
    dynamic_params[key] = value;
    if (logging_enabled)
        std::cout &lt;&lt; &quot;[LOG] Dynamic param set: &quot; &lt;&lt; key &lt;&lt; &quot;
}

double getDynamicParameter(const std::string&amp; key) const {
    auto it = dynamic_params.find(key);
    if (it != dynamic_params.end()) return it-&gt;second;
    if (logging_enabled)
        std::cout &lt;&lt; &quot;[LOG] Dynamic param not found: &quot; &lt;&lt; key &lt;&lt; s
    return std::numeric_limits&lt;double&gt;::quiet_NaN();
}

void exportState(const std::string&amp; filename = &quot;UQFFLearningAssessment_state.txt&quot
    std::ofstream ofs(filename);
    if (!ofs) {
        std::cerr &lt;&lt; &quot;Error: Cannot open &#39;&quot; &lt;&lt; filename &lt;&lt; &qu
        return;
    }
    ofs &lt;&lt; &quot;# UQFFLearningAssessment (Research Student Edition) state export\n&quot
    ofs &lt;&lt; std::scientific &lt;&lt; std::setprecision(10);
    const char* names[] = {
        &quot;G&quot;,&quot;c_light&quot;,&quot;hbar&quot;,&quot;Lambda&quot;,&quot;f_TRZ&quot
        &quot;diversity_score&quot;,&quot;dynamic_score&quot;,&quot;scalability_score&quot;,&q
        &quot;M_sgr1745&quot;,&quot;r_sgr1745&quot;,&quot;B_sgr1745&quot;,&quot;D0_sgr1745&quo
        &quot;M_sgr0501&quot;,&quot;r_sgr0501&quot;,&quot;B_sgr0501&quot;,&quot;D0_sgr0501&quo
        &quot;M_smbh&quot;,&quot;r_smbh&quot;,&quot;M_dot_smbh&quot;,&quot;spin_smbh&quot;,&qu
        &quot;M_tapestry&quot;,&quot;r_tapestry&quot;,&quot;rho_wind_tap&quot;,&quot;v_wind_ta
        &quot;M_wd2&quot;,&quot;r_wd2&quot;,&quot;rho_wind_wd2&quot;,&quot;v_wind_wd2&quot;,&q
        &quot;M_pillars&quot;,&quot;r_pillars&quot;,&quot;E_0_pillars&quot;,&quot;rho_wind_pil
        &quot;M_rings&quot;,&quot;r_rings&quot;,&quot;Hz_rings&quot;,&quot;L_factor_rings&quot
    };
    for (const char* n : names)
        ofs &lt;&lt; n &lt;&lt; &quot; = &quot; &lt;&lt; getVariable(n) &lt;&lt; &quot;\n&quot
    for (const auto&amp; kv : dynamic_params)
        ofs &lt;&lt; &quot;dynamic.&quot; &lt;&lt; kv.first &lt;&lt; &quot; = &quot; &lt;&lt;
    if (logging_enabled)
        std::cout &lt;&lt; &quot;[LOG] State exported to: &quot; &lt;&lt; filename &lt;&lt; st
}

// Cross-validate g_base[sys_idx] against another UQFF module&#39;s g at the same time
// Store the other module&#39;s g value via setDynamicParameter(&quot;cross_g&quot;, value) fi
template&lt;typename OtherModuleT&gt;
double cross_validate(const OtherModuleT&amp; /*other*/, int sys_idx,
                      double /*t_seconds*/ = 0.0) const {
    double g_here  = compute_g_base(sys_idx);
    double g_other = 0.0;
    auto it = dynamic_params.find(&quot;cross_g&quot;);
    if (it != dynamic_params.end()) g_other = it-&gt;second;
```

```
781         double frac = (g_here != 0.0)
782                       ? std::abs(g_other - g_here) / std::abs(g_here)
783                       : std::numeric_limits&lt;double&gt;::quiet_NaN();
784         if (logging_enabled)
785             std::cout &lt;&lt; &quot;[XVAL] sys[&quot; &lt;&lt; sys_idx &lt;&lt; &quot;] g_here=&qu
786                       &lt;&lt; &quot;  g_other=&quot; &lt;&lt; g_other
787                       &lt;&lt; &quot;  frac_diff=&quot; &lt;&lt; frac &lt;&lt; std::endl;
788         return frac;
789     }
790 };
791
792 #endif // UQFF_LEARNING_ASSESSMENT_H
```

— End of UQFFLearningAssessment_001.cpp —